

AI QUANT-CODER

Intelligent Stock Predictor & Code Assistant — end-to-end framework linking forecasts to production-ready trading code. Submission: AMD Developer Hackathon (Act II).

TEAM: AGHA ESSA KHAN & MISBAH RAMZAN

Challenge & Vision

Problem

Forecasts rarely translate directly into executable strategy code; building production bots demands high expertise and time.

Vision

End-to-end framework that binds predictions to automated coding, using local LLMs on AMD GPUs for secure, zero-quota development.

Impact

Fast, secure, customizable pipelines for quantitative developers — from signal to deployable script.

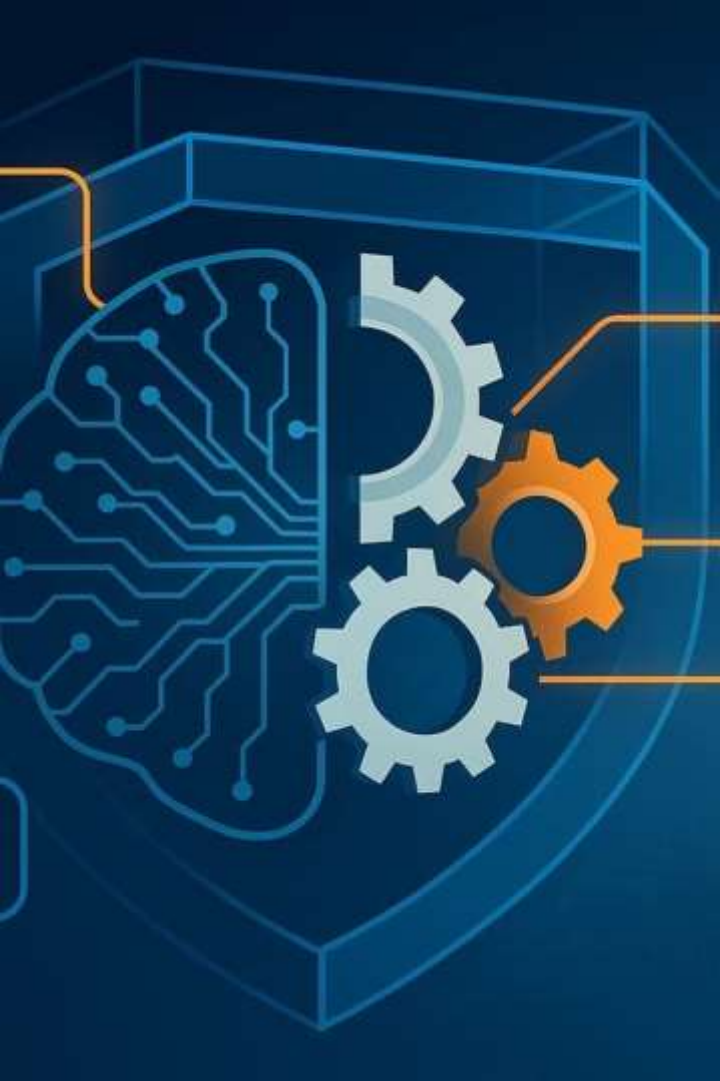


Core System Architecture



Modular Pipelines

- Data loading & technical analytics (yfinance; SMA/EMA/RSI)
- Predictive forecast engine (ARIMA-based 30-day projections)
- TF-IDF RAG database of trading scripts (NumPy index)
- Stream-enabled LLM client for context-aware code generation



Market Predictor Module

1

Data

Auto-fetch up to 10 years of historical prices for robust training.

2

Modeling

ARIMA(1,1,0) for non-stationary series; produces 30-day forecast with confidence intervals.

3

Signals

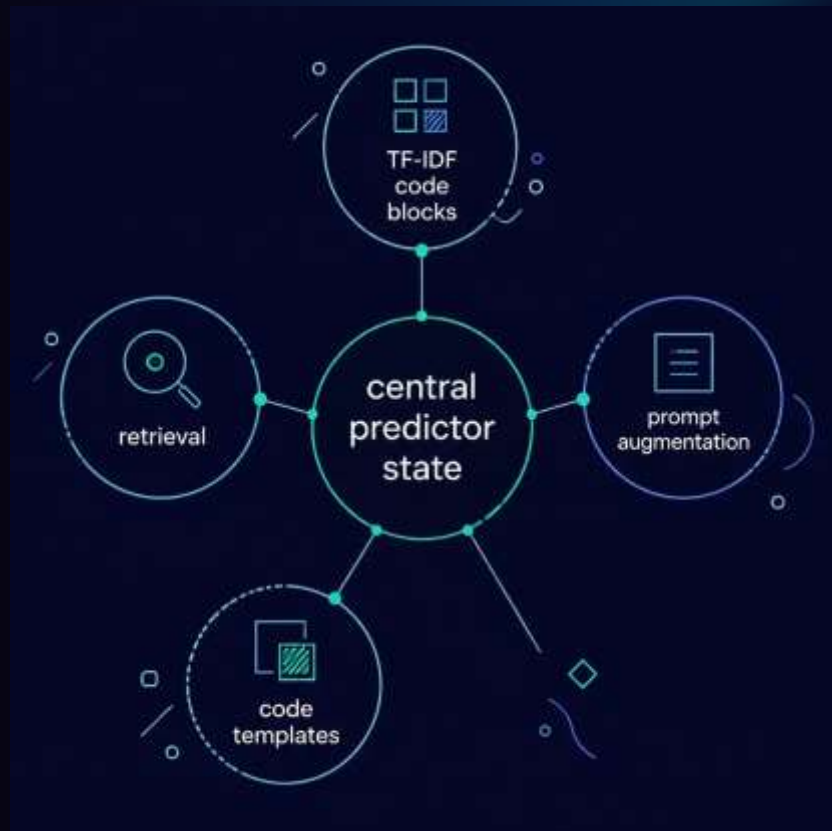
SMA(20/50) crossovers and RSI-14 to flag momentum and overbought/oversold states.

4

Visualization

Interactive dark-mode Plotly charts: historic close, MAs, and prediction bands.

RAG Assistant Module



How it Works

- TF-IDF index of local Python trading scripts segmented by functions/classes.
- Cosine similarity retrieval yields relevant template code blocks.
- Predictor state (ticker, RSI, forecast, model) merged into system prompt.
- Augmented prompts maximize structural correctness for generated code.

Generative Pipeline & UX Innovations



Thread-Isolated Workers

Background generator threads keep Streamlit responsive during long runs.



Hot-Swappable Layouts

Dynamic panel swapping (Graph ↔ Chat) for focused workflows.



Safe State & Zero Lock

Clean session init and multi-session threads to prevent UI blocking and preserve history.



CPU Vs GPU Server Speed



Scale with io.net's decentralized GPU comp

AMD GPU & ROCm Production Stack

Local Prototyping

Ollama CPU/APU setups for quick development.

Production

AMD Developer Cloud with MI210/MI250/MI300 GPUs and ROCm 6.x for vLLM serving.

Serving

vLLM + PagedAttention for high-throughput inference; OpenAI-compatible endpoints for Streamlit integration.

Security

Local models reduce external quotas and protect proprietary code.



Roadmap: Execution & Scaling

1

Sandboxed Execution

Isolated Docker runtime to safely execute generated Python and show backtest plots.

2

Multi-Model Forecasting

Add LSTM and Prophet alongside ARIMA for model choice and ensemble forecasts.

3

Real-Time Integrations

Paper trading and direct deployment to broker APIs for live strategy execution.

4

Advanced Vector DBs

Integrate ChromaDB/FAISS for massive script datasets and faster retrieval.

Hackathon Innovations — Competitive Edge

Performance

ROCm + vLLM ensures institutional throughput for large models.

Usability

Streamlined UX with layout controls and responsive generation threads.

Security

Local LLMs and on-prem vectors eliminate external quota risks.

Why This Wins

- Direct mapping from predictive signals to production code – unique end-to-end value.
- AMD-optimized stack delivers competitive throughput and scalability.
- Designed for quant developers: fast iteration, reproducible templates, safe execution.

Thank You — AI Quant-Coder

Empowering traders with predictive automated coding. Developed by Agha Essa Khan & Misbah Ramzan. Ready for competition deployment and live demos.

